

Module 4: Operators

4.1 Arithmetic Operators

- Addition (+): adds two values
Example: `int sum = 5 + 3;` (result is 8)
- Subtraction (-): subtracts the right operand from the left
Example: `int diff = 10 - 4;` (result is 6)
- Multiplication (*): multiplies two values
Example: `int product = 6 * 7;` (result is 42)
- Division (/): divides left operand by right operand
Integer division: when both operands are integers, the decimal part is discarded
Example: `int result = 7 / 2;` (result is 3, not 3.5)
Floating point division: if at least one operand is a float or double, the result is decimal
Example: `double result = 7.0 / 2;` (result is 3.5)
- Modulus (%): gives the remainder after integer division
 - Example: `int rem = 10 % 3;` (result is 1 because $10 = 3 \times 3 + 1$)
 - Only works with integer operands
 - Useful for checking if a number is even or odd (`number % 2 == 0` means even)
- Operator precedence (order of evaluation, from highest to lowest)
 - Parentheses ()
 - Multiplication, division, modulus: *, /, %
 - Addition and subtraction: +, -
 - When operators have the same precedence, they evaluate left to right
- Using parentheses to change evaluation order
 - Example: `2 + 3 * 4` evaluates as `2 + 12 = 14`
 - Example: `(2 + 3) * 4` evaluates as `5 * 4 = 20`

4.2 Assignment Operators

- Simple assignment (=): assigns the value on the right to the variable on the left
- `x = 10;` assigns 10 to x
- The right side is evaluated first, then stored in the left side
- Compound assignment operators (shorthand forms)
- `+=` (add and assign): `x += 5` is the same as `x = x + 5`
- `-=` (subtract and assign): `x -= 3` is the same as `x = x - 3`
- `*=` (multiply and assign): `x *= 2` is the same as `x = x * 2`
- `/=` (divide and assign): `x /= 4` is the same as `x = x / 4`
- `%=` (modulus and assign): `x %= 7` is the same as `x = x % 7`

4.3 Increment and Decrement Operators

- Increment operator (++): adds 1 to a variable
- Pre-increment (++x): increments x first, then uses the new value in the expression

- Post-increment (x++): uses the current value of x in the expression, then increments
- Example to show the difference:
 - `int x = 5; int a = ++x; (x becomes 6, a is 6)`
 - `int x = 5; int b = x++; (b is 5, then x becomes 6)`
- Decrement operator (--): subtracts 1 from a variable
- Pre-decrement (--x): decrements first, then uses the new value
- Post-decrement (x--): uses the current value, then decrements
- These operators are very commonly used in loops

4.4 Relational (Comparison) Operators

- Used to compare two values and produce a boolean result (true or false)
- Equal to (==)
- Tests if two values are equal
- Very common mistake: using = (assignment) instead of == (comparison)
- Example: `if (x == 10)` checks if x equals 10
- Not equal to (!=)
- True when the two values are different
- Greater than (>): true if left is larger than right
- Less than (<): true if left is smaller than right
- Greater than or equal to (>=): true if left is larger than or equal to right
- Less than or equal to (<=): true if left is smaller than or equal to right
- The result of any comparison is either 1 (true) or 0 (false)

4.5 Logical Operators

- Used to combine multiple conditions
- Logical AND (&&)
- Result is true only if BOTH conditions are true
- Example: `if (age >= 18 && age <= 65)`
- If the first condition is false, the second is not evaluated (short-circuit)
- Logical OR (||)
- Result is true if AT LEAST ONE condition is true
- Example: `if (grade == 'A' || grade == 'B')`
- If the first condition is true, the second is not evaluated (short-circuit)
- Logical NOT (!)
- Reverses the boolean value
- `!true` is false, `!false` is true
- Example: `if (!isLoggedIn)` means "if the user is not logged in"
- Short-circuit evaluation
- In &&: if the first condition is false, C++ skips the second condition entirely
- In ||: if the first condition is true, C++ skips the second condition entirely

- This can prevent errors like dividing by zero in a compound condition

4.6 Conditional (Ternary) Operator

- Provides a compact way to write a simple if-else decision
- Syntax: `condition ? value_if_true : value_if_false`
- Example: `int max = (a > b) ? a : b;`
- If `a > b` is true, max gets the value of a; otherwise max gets the value of b
- Can be used directly inside a cout statement
- `cout << (x % 2 == 0 ? "Even" : "Odd");`
- Limitation: should only be used for simple choices; nested ternary operators are hard to read

4.7 Type Casting

- Sometimes a value of one type needs to be treated as a different type
 - Implicit conversion (automatic): the compiler converts automatically when safe
- int to double: `int x = 5; double y = x; (y becomes 5.0)`
- char to int: `char c = 'A'; int n = c; (n becomes 65)`
- Explicit conversion (casting): the programmer forces the conversion
 - C-style cast: `(int)3.7` gives 3 (truncates, does not round)
 - C++ style cast: `static_cast<int>(3.7)` gives 3
 - `static_cast` is preferred in C++ because it is clearer and safer
 - Data loss during conversion
 - Casting double to int discards the decimal part
 - Casting a large long long to int can overflow